

Montaje y Calibración de sensores ambientales low-cost



Temario

- Modelo de Sistema Tecnológico
- Fundamentos de Electricidad y Electrónica
- Sensores y Actuadores
- Arquitectura y Programación en Arduino
- Montaje de prototipos repetibles y modificables con sensores low-cost y adquisición de datos.

Objetivos

Desarrollar capacidades técnicas para diseñar, construir y calibrar prototipos de monitoreo ambiental de bajo costo, utilizando plataformas open-source (Arduino) y sensores accesibles.

- **Comprender** la arquitectura de sistemas tecnológicos (input-procesamiento-output)
- **Dominar** fundamentos básicos de la electrónica aplicada.
- **Programar** microcontroladores Arduino para adquisición de datos.
- **Construir** 2 prototipos ambientales low-cost repetibles

Introducción a la Programación y la Electrónica

por Nicolás Saganías

[LinkedIn](#)

[Upwork](#)

nicolassaganias@protonmail.com

Índice

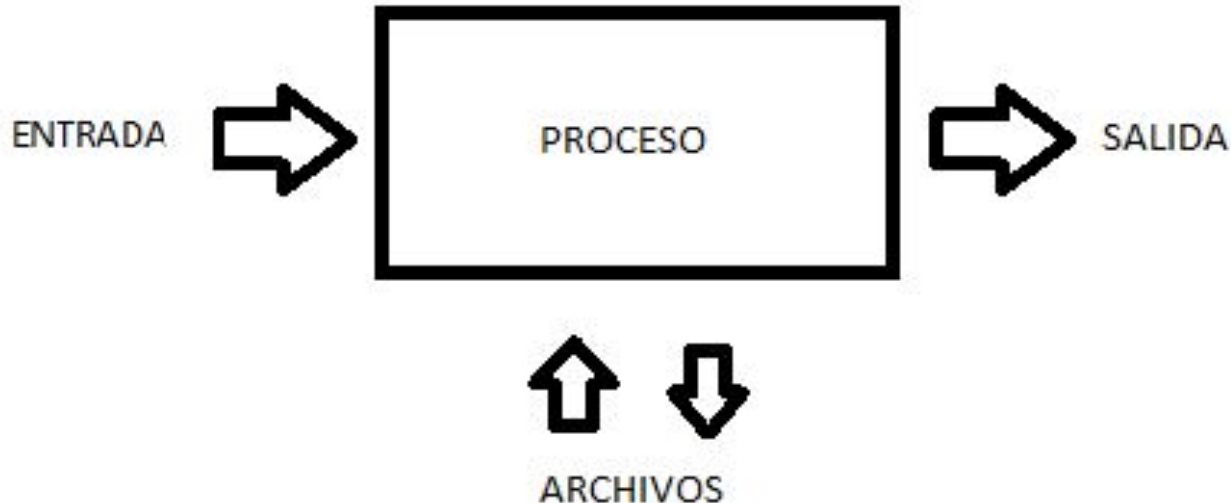
- Modelo de Sistema Tecnológico
- Fundamentos de Electricidad y Electrónica
- Sensores y Actuadores
- Arquitectura Arduino
- Programación en Arduino
- Programación de sensores y kit base para los prototipos

¿Cómo una máquina entiende el mundo físico?

- Entrada → Procesamiento → Salida
- Sensor → Microcontrolador → Actuador

Sistema

- Los **sistemas** (por ejemplo, máquinas) que aplican tecnología tomando una **entrada**, cambiándola de acuerdo con el uso del sistema y luego produciendo un **resultado** se denominan **sistemas tecnológicos**.



Periféricos: INPUTS y OUTPUT



Periféricos: INPUTS y OUTPUT



The System:

Inputs:

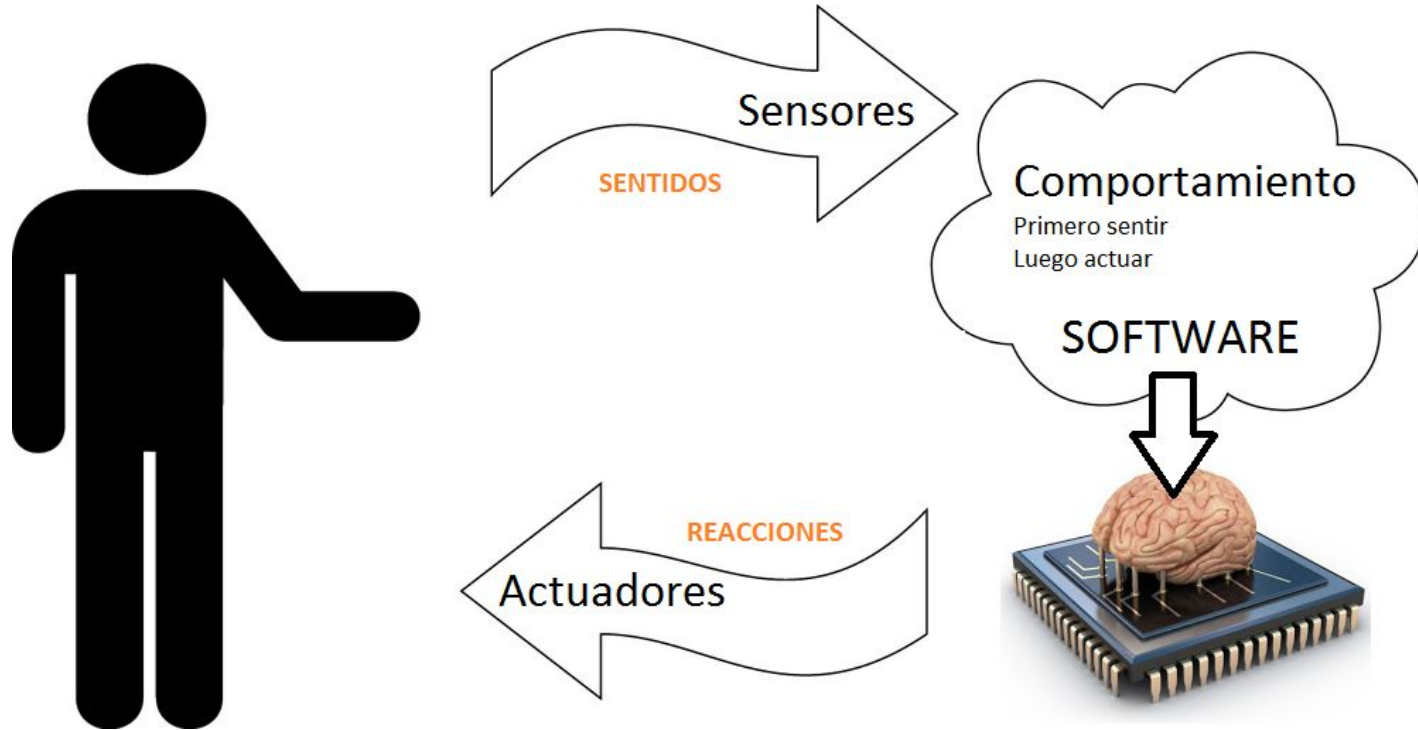


Process:

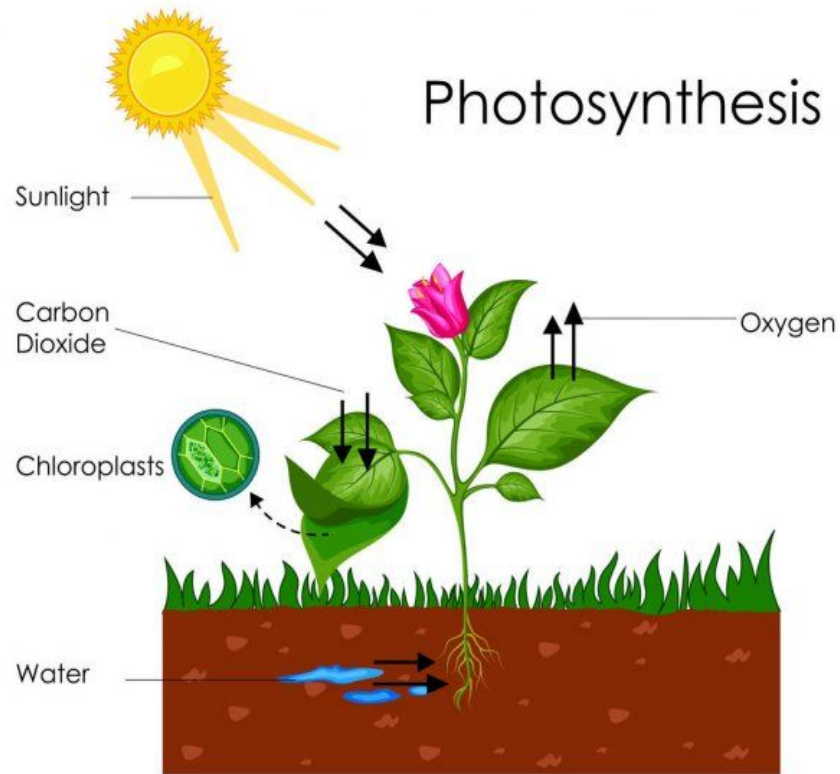
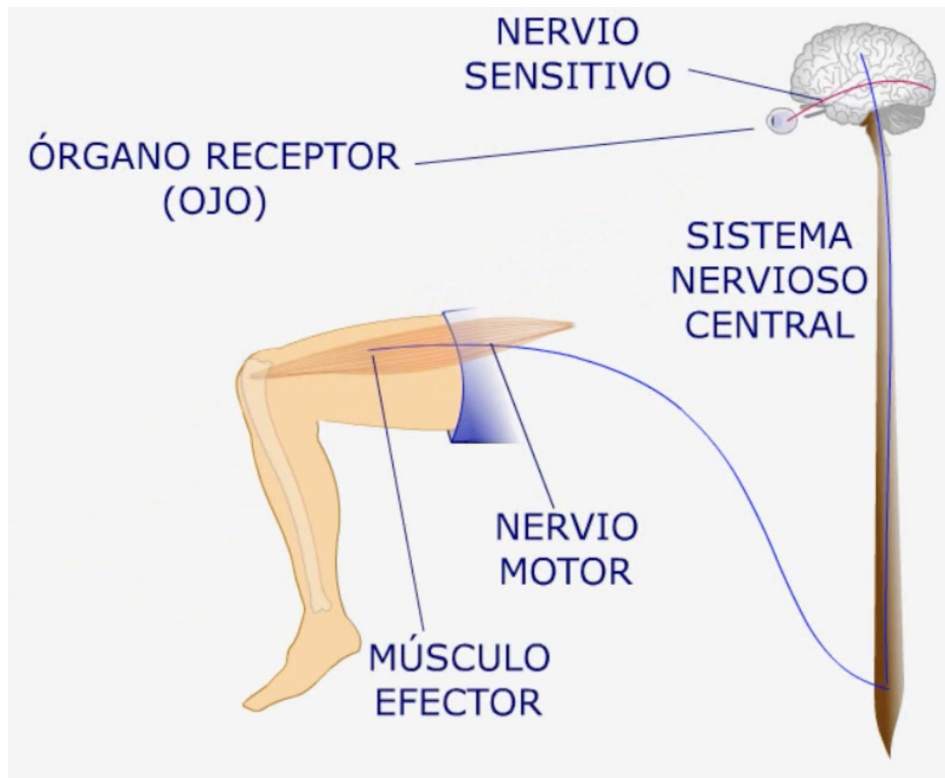


Outputs:

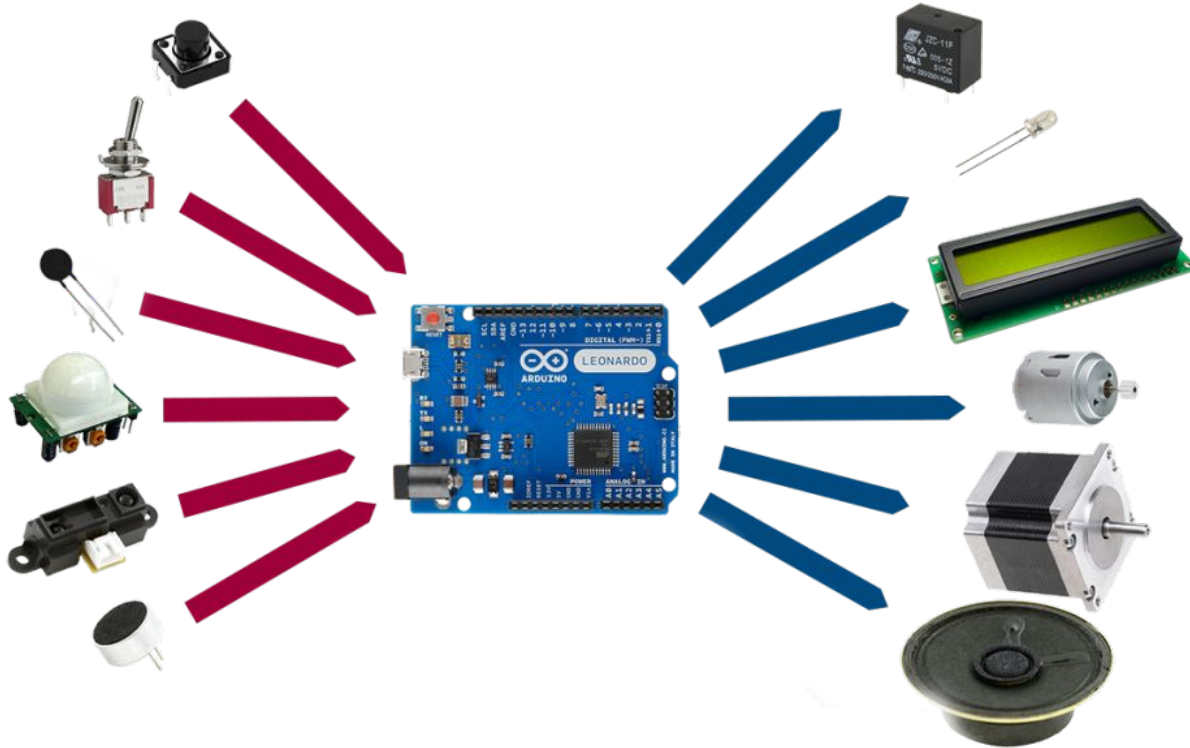
Sensores y Actuadores



Sensores/Actuadores Naturales



Universo Arduino

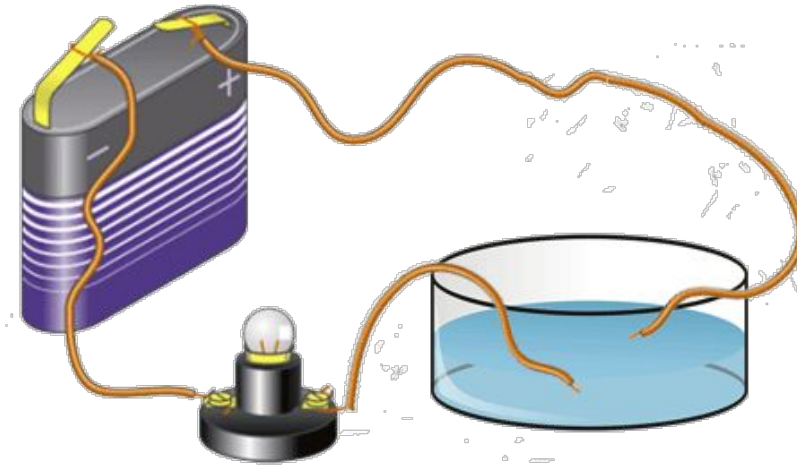


Sensores

Actuadores

Electricidad

- La **electricidad** (del griego *élektron*, cuyo significado es 'ámbar') es el conjunto de fenómenos físicos relacionados con la presencia y flujo de **cargas eléctricas**.
- En la naturaleza se manifiesta en una gran variedad de fenómenos como los **rayos**, la **electricidad estática**, la **inducción electromagnética** o el flujo de **corriente eléctrica**.



CORRIENTE CONTINUA



VS

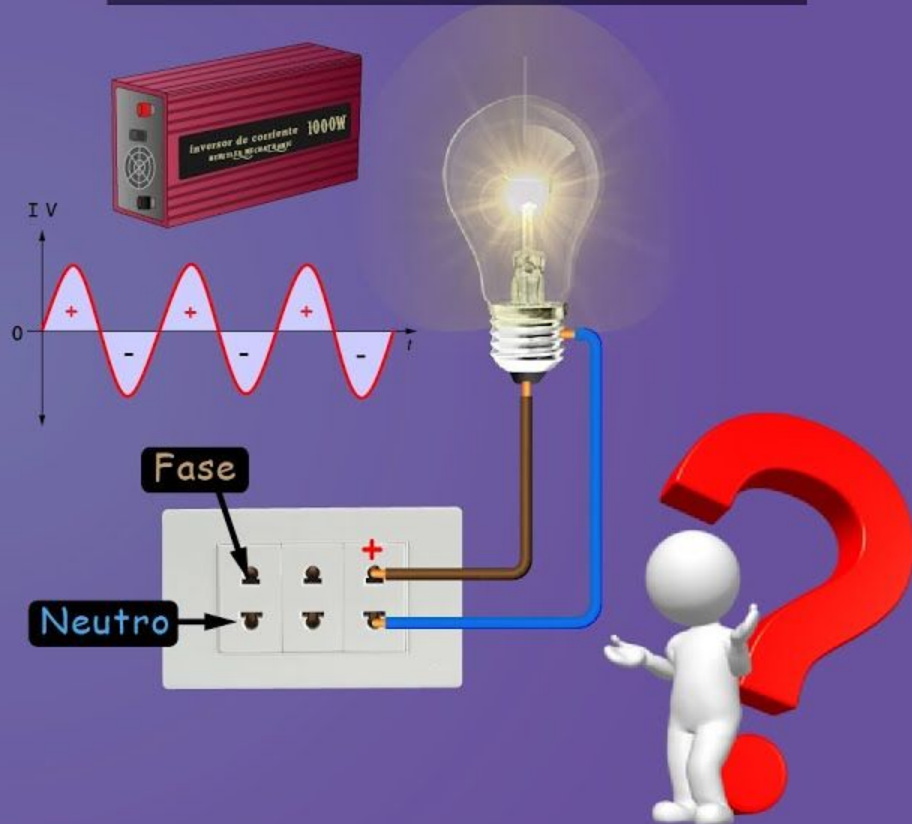
CORRIENTE ALTERNA



Corriente Directa (DC)

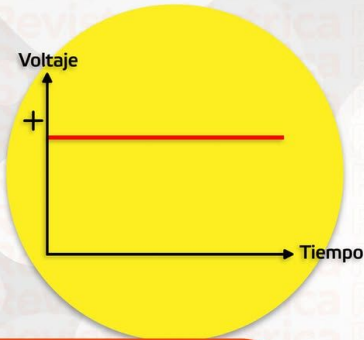


Corriente Alterna (AC)



Diferencias

Corriente Continua y Corriente Alterna

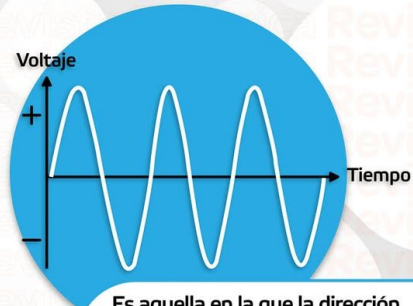


Es la corriente eléctrica que fluye de forma constante en una dirección.

Mantiene constante su polaridad y la energía fluye en el mismo sentido, sin importar la magnitud o voltaje.

Sirve en aplicaciones y aparatos que poseen un voltaje pequeño

Es producida por pilas o por células fotovoltaicas.



Es aquella en la que la dirección del flujo de electrones va y viene en intervalos regulares o en ciclos.

La magnitud y dirección varían cíclicamente.

No tiene polaridad.

Se puede aumentar o disminuir el voltaje o tensión.

Se genera en centrales eléctricas y mediante el uso de alternadores.

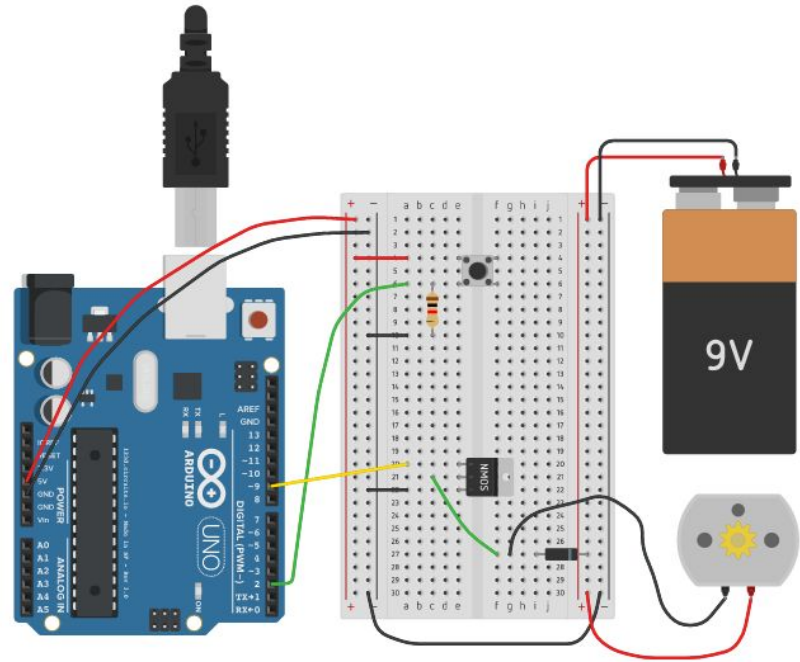
Con ella funcionan muchos electrodomésticos y sirve para suministrar de electricidad a los hogares.



Corriente Continua en Electrónica y Arduino

Arduino usa DC porque sus circuitos trabajan con:

- **Voltaje bajo, estable y seguro.**
- Lógica de funcionamiento 0 y 1, 0v y 5v.
- Porque es **almacenable / transportable.**
- Comodidad **USB.**



Arduino

Arduino es una compañía de desarrollo de **software y hardware libres**, así como una **comunidad internacional** que diseña y manufactura placas de desarrollo de hardware para **construir dispositivos digitales** que puedan detectar y controlar objetos del mundo real.

Placa Arduino UNO



```
19
20 This example code is in the public domain.
21
22 http://www.arduino.cc/en/Tutorial/Blink
23
24 */
25 // the setup function runs once when you press reset or power the board
26 void setup() {
27   // initialize digital pin LED_BUILTIN as an output.
28   pinMode(LED_BUILTIN, OUTPUT);
29   Serial.begin(9600);
30 }
31
32 // the loop function runs over and over again forever
33 void loop() {
34   digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
35   delay(1000); // wait for a second
36   digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW
37   delay(1000); // wait for a second
38   Serial.println("LED is blinking");
39 }
40
```

Output Serial Monitor x

Message (Ctrl+Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem143201')

LED is blinking
LED is blinking

New Line 9600 baud

31 UTF-8 Arduino Nano 33 BLE on /dev/cu.usbmodem143201

DIY DIT DIN

Do it yourself. Do it together. Do it now.

In the punk subculture, the DIY ethic is tied to punk ideology and anticonsumerism, as a rejection of the need to purchase items or use existing systems or processes. Arguably since the 1970s, emerging punk bands began to record their music, produce albums and merchandise, distribute their works and often performed basement shows in residential homes rather than at traditional venues, to avoid corporate sponsorship or to secure freedom in performance. Since many venues tend to shy away from more experimental music, houses and other private venues are often the only places at which these bands can play.

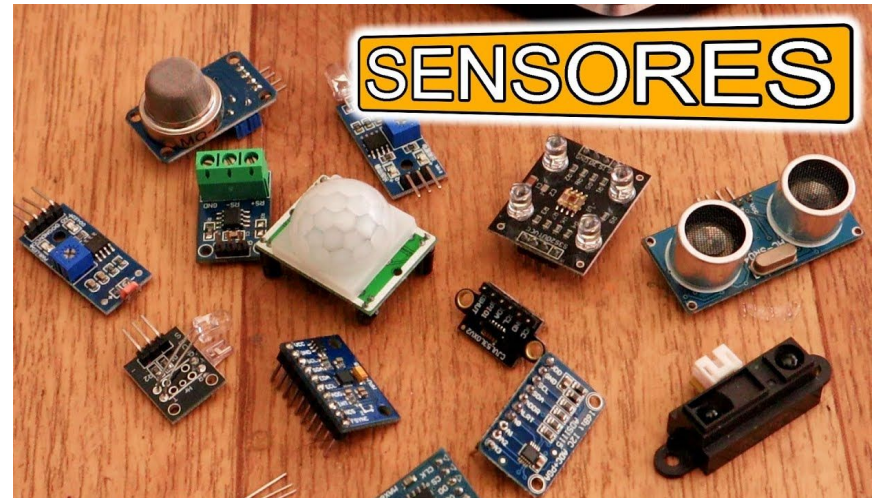


“Al principio, había transistores, sensores ambientales rudimentarios alojados en cajas de madera, y lápiz y papel para registrar los datos. Hoy en día, programamos microcontroladores y fabricamos nuestros propios sensores ambientales para transmitir un torrente de datos en tiempo real a portales web para que el mundo los vea — ¡por una fracción del costo de aquellas cajas de madera!”

<https://www.envirodiy.org/wide-wide-world-of-diy-and-dit/>

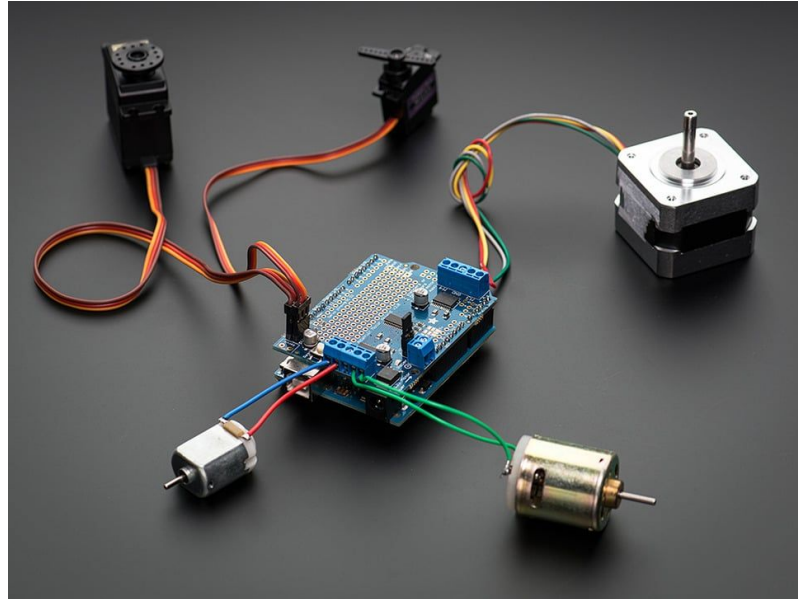
Sensores

Un sensor es un dispositivo electrónico que **recoge datos analógicos del mundo real y los transforma en datos digitales** con los que alimenta de información a Arduino.



Actuadores

Un actuador es un dispositivo que **recibe señales digitales de Arduino y las transforma en acción física** sobre un elemento externo del mundo real.



Sensores y Actuadores: Energía y Data

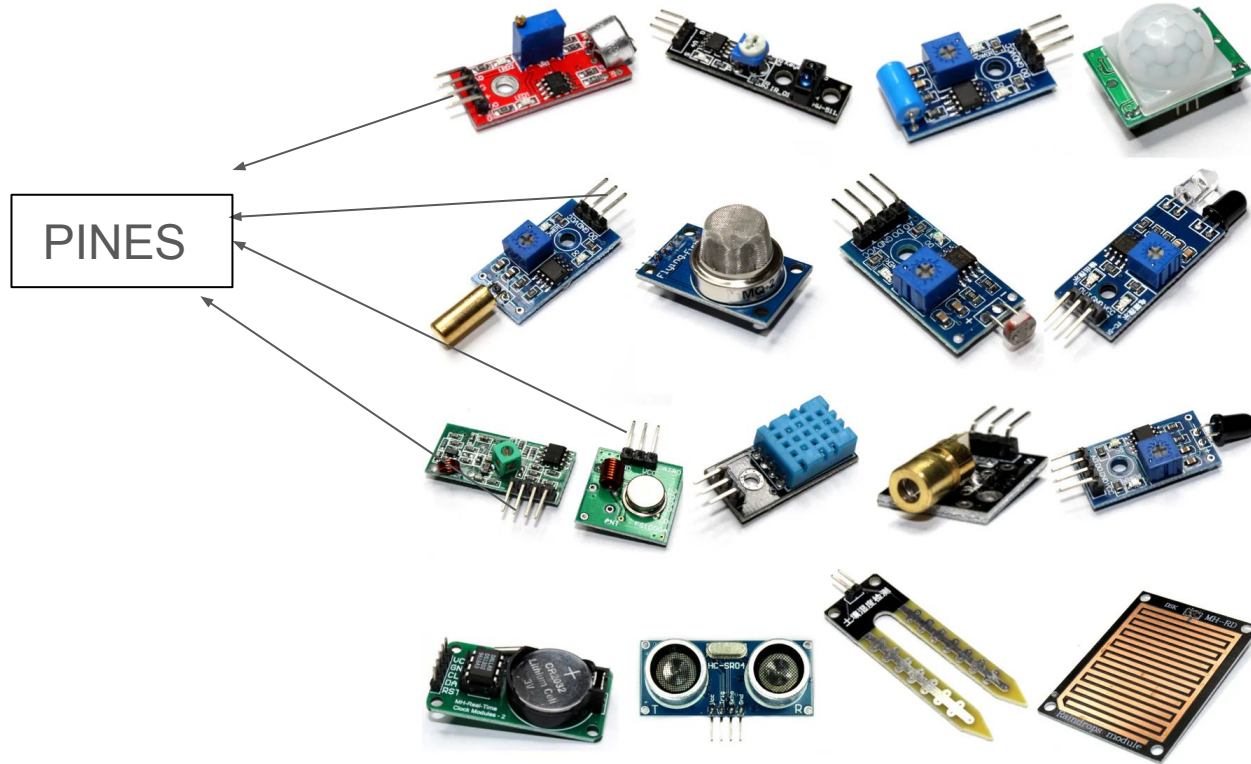
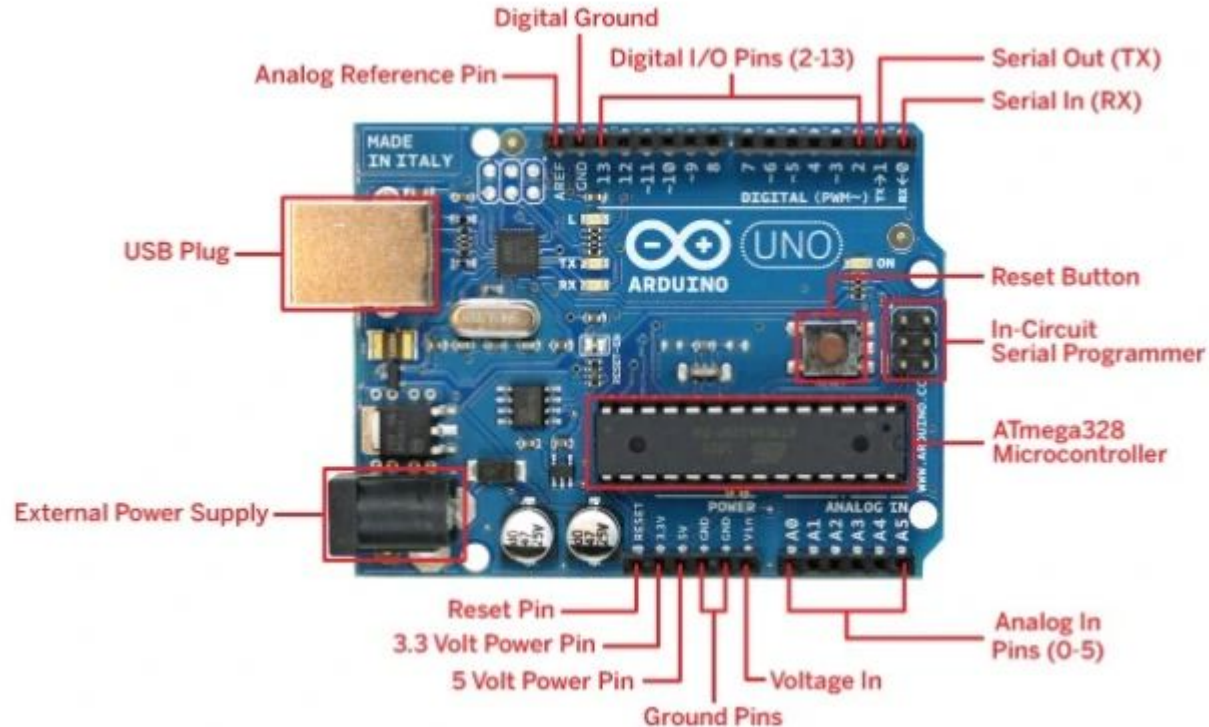


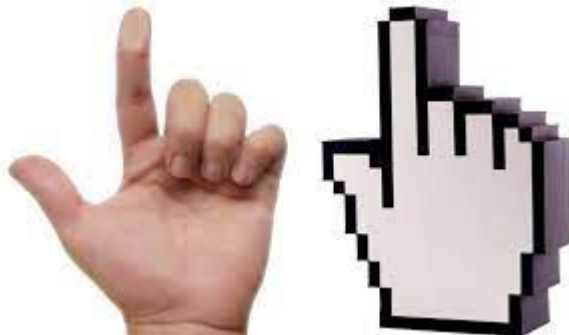
Diagrama de pines de entradas y salidas de Arduino



Conversión Analógico / Digital

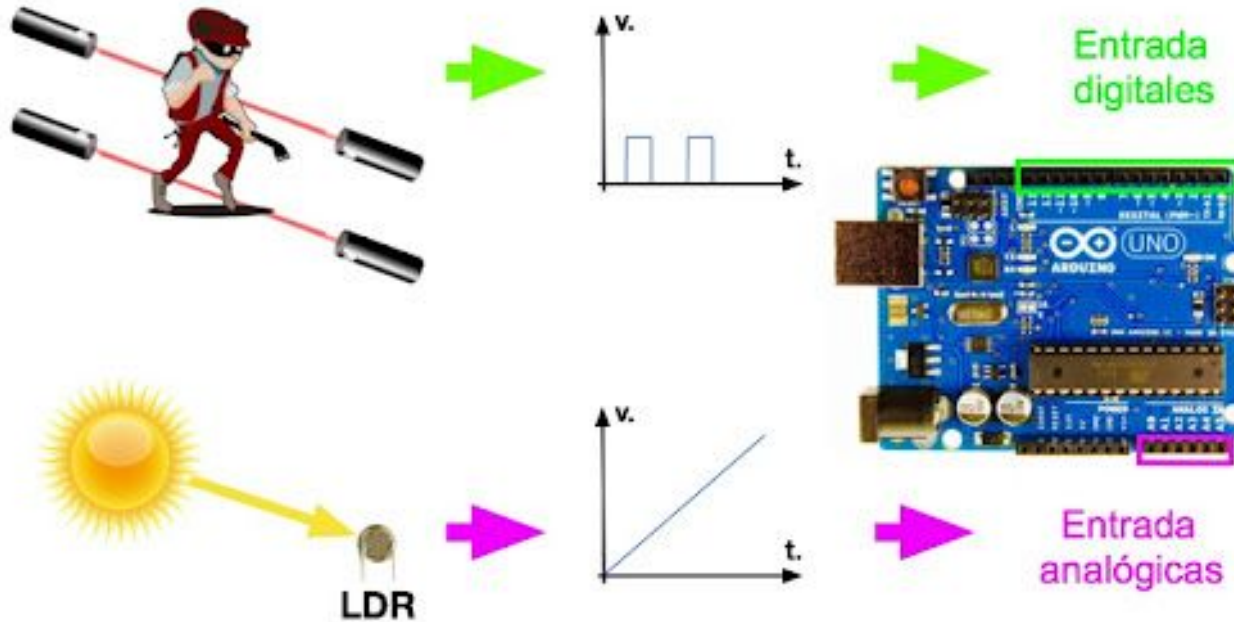


La señal analógica tiene valores continuos

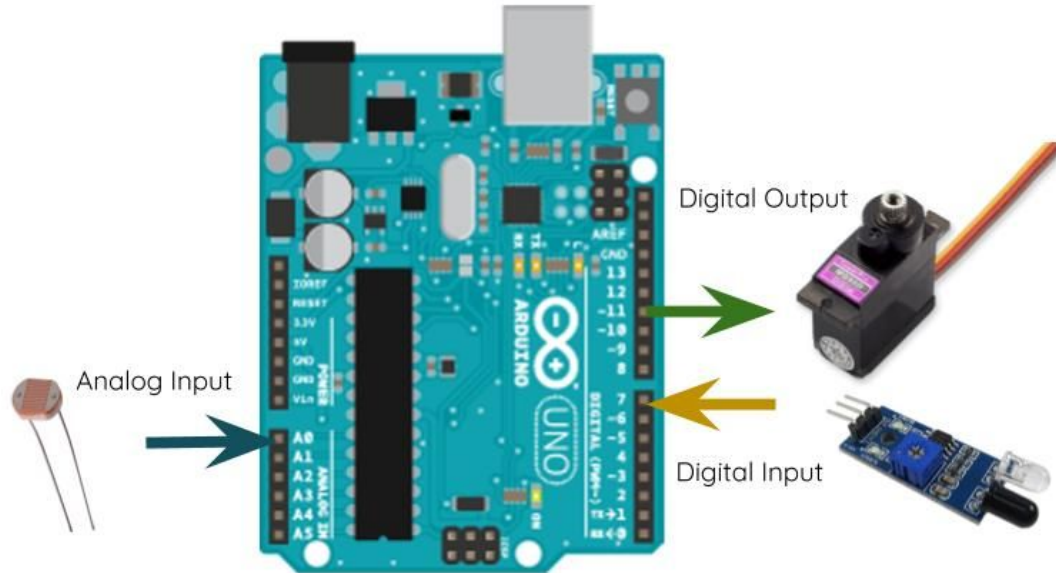


La señal digital tiene valores discretos

Ejemplo Entrada Digital y Analógica



Entradas y Salidas Analógicas y Digitales



Programación

Un **lenguaje de programación** le proporciona a una persona, la capacidad de dar instrucciones a un sistema informático.

```
package main

import "fmt"

func main() {
    fmt.Printf("hello, world")
}
```

Lenguaje de alto nivel que
entiende el programador



```
0101010111101110001101
0100010100010101001010
0101010010101010000101
0011010001010100011110
0110010100101010101001
1110001101010010010001
```

Lenguaje de máquina que
entiende el procesador

Programación en Arduino

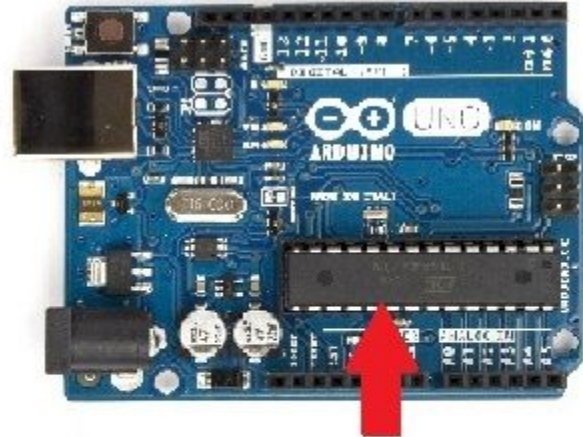
Trabajar con un Arduino consiste fundamentalmente en **interactuar con los diferentes puertos de entrada y salida** del Arduino.



Notebook corriendo
Arduino IDE para poder
programar Arduino



Cable de conexión, para
transferir el programa y
para alimentar
momentáneamente la
placa Arduino



Microcontrolador, que
adentro tiene memoria
FLASH que guarda el
programa

ARDUINO IDE (Software)

Verificar/Compilar

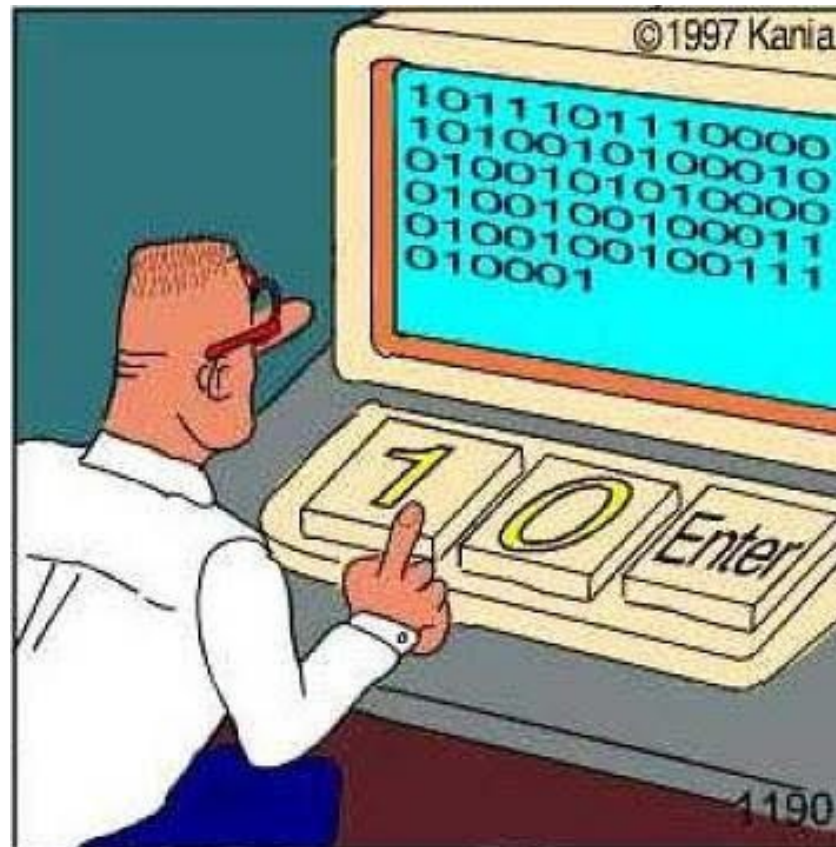
Cargar/Subir

Monitor Serial



Información
placa y puerto

A Programar!



- Conectar Arduino por USB
- Abrir Arduino IDE
- Abrir Ejemplo Blink.ino

Hola Mundo!

Blink.ino

```
1 // the setup function runs once when you press reset or power the board
2 void setup() {
3     // initialize digital pin LED_BUILTIN as an output.
4     pinMode(LED_BUILTIN, OUTPUT);
5 }
6
7 // the loop function runs over and over again forever
8 void loop() {
9     digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
10    delay(1000); // wait for a second
11    digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW
12    delay(1000); // wait for a second
13 }
14
```

Resultado esperado

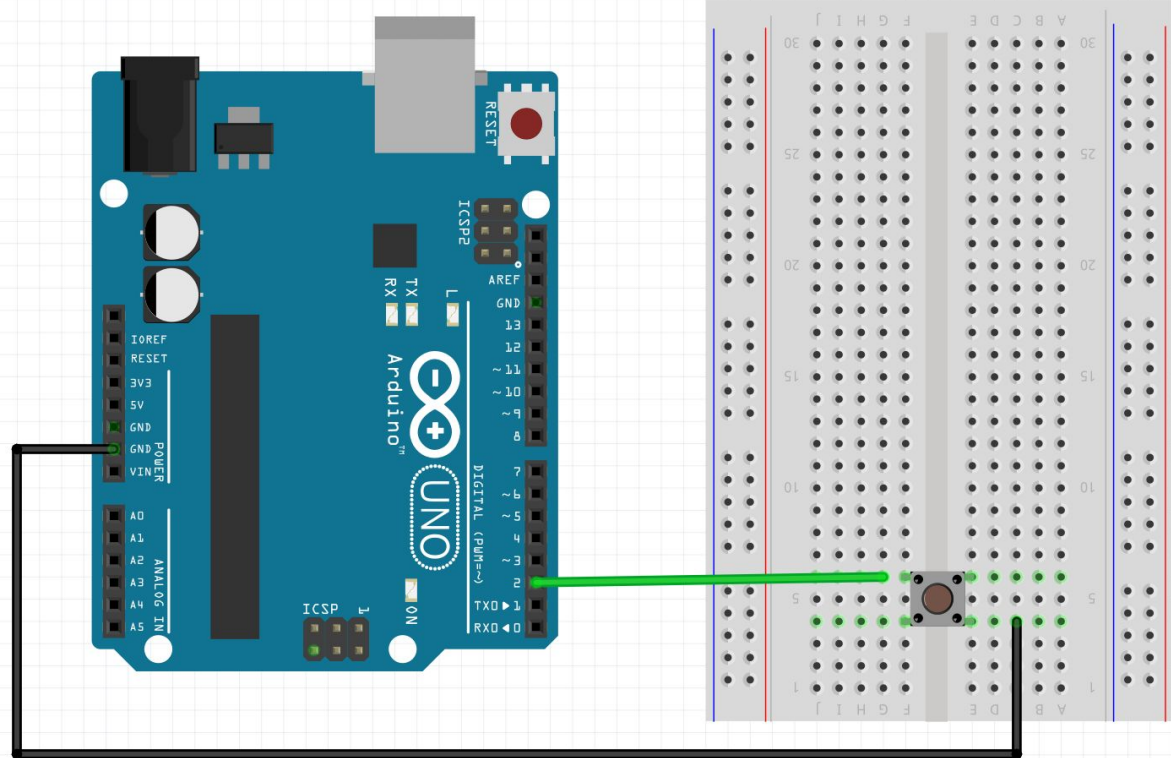


Ejemplo Digital Read: Conexión a Protoboard

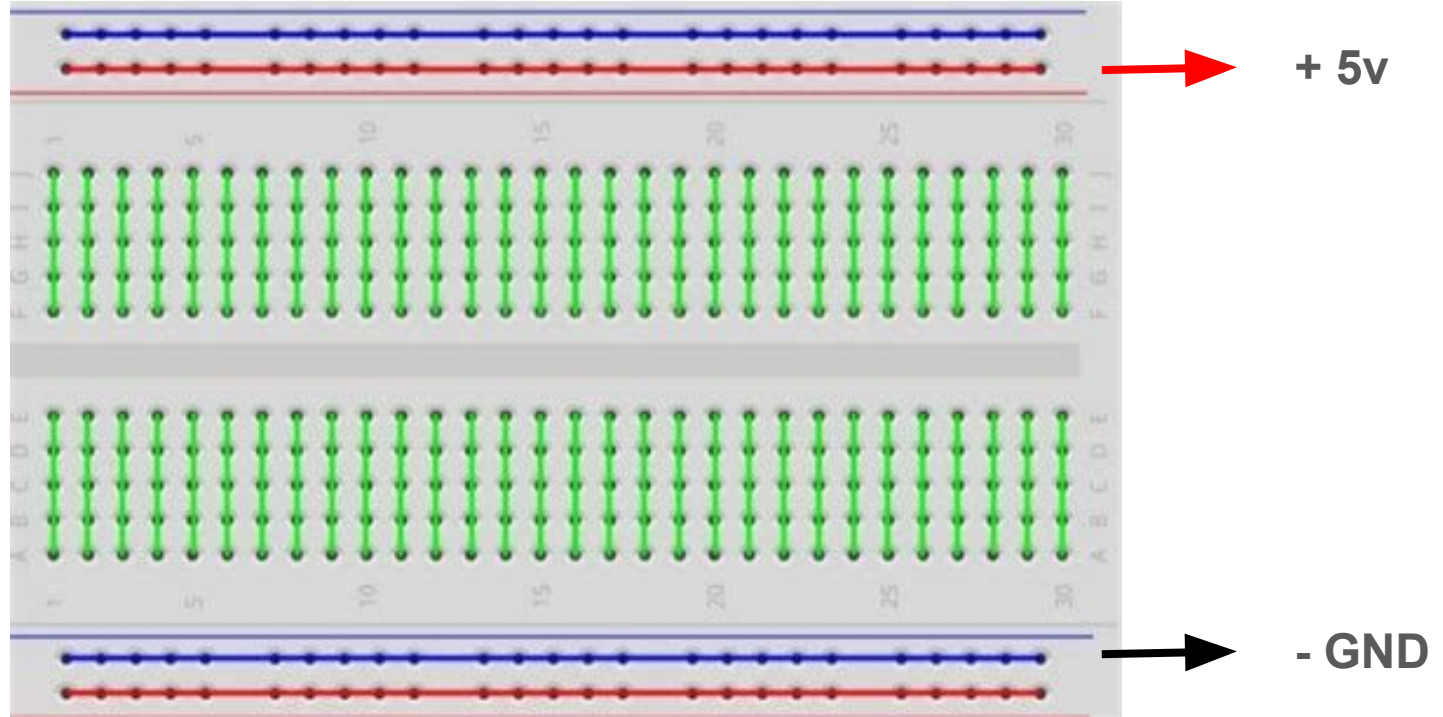
Conexiones:

GND - Botón

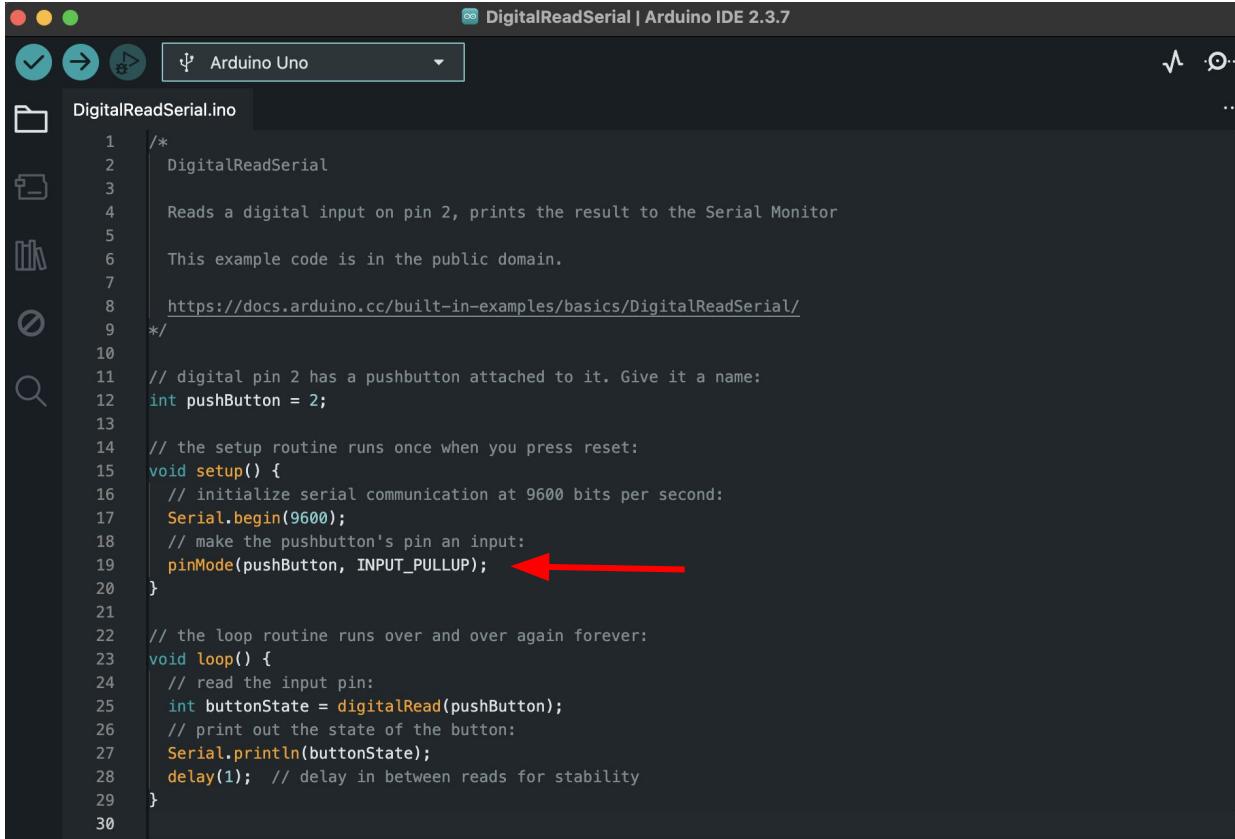
Pin 2 - Botón



Funcionamiento Protoboard



Ejemplos Digital Read: Programación



```
1  /*
2  DigitalReadSerial
3
4  Reads a digital input on pin 2, prints the result to the Serial Monitor
5
6  This example code is in the public domain.
7
8  https://docs.arduino.cc/built-in-examples/basics/DigitalReadSerial/
9  */
10
11 // digital pin 2 has a pushbutton attached to it. Give it a name:
12 int pushButton = 2;
13
14 // the setup routine runs once when you press reset:
15 void setup() {
16   // initialize serial communication at 9600 bits per second:
17   Serial.begin(9600);
18   // make the pushbutton's pin an input:
19   pinMode(pushButton, INPUT_PULLUP);
20 }
21
22 // the loop routine runs over and over again forever:
23 void loop() {
24   // read the input pin:
25   int buttonState = digitalRead(pushButton);
26   // print out the state of the button:
27   Serial.println(buttonState);
28   delay(1); // delay in between reads for stability
29 }
30
```

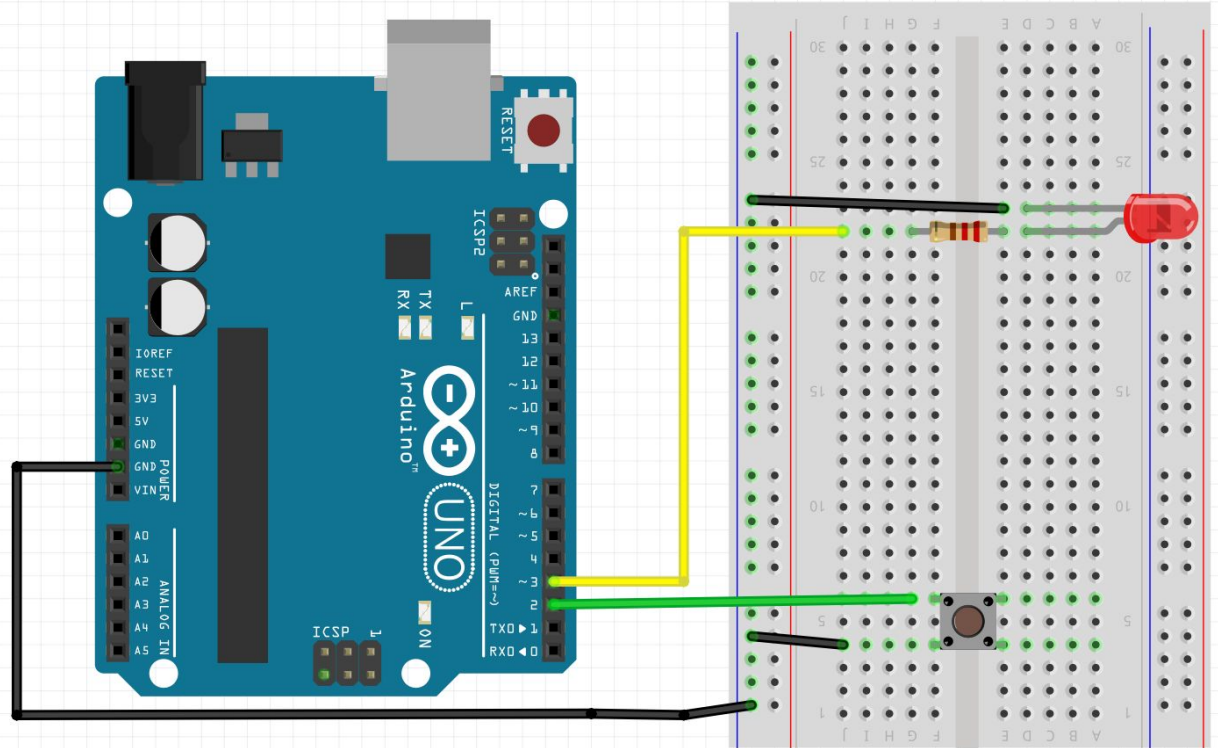
Ejemplo Arduino + LED + Botón

Conexiones:

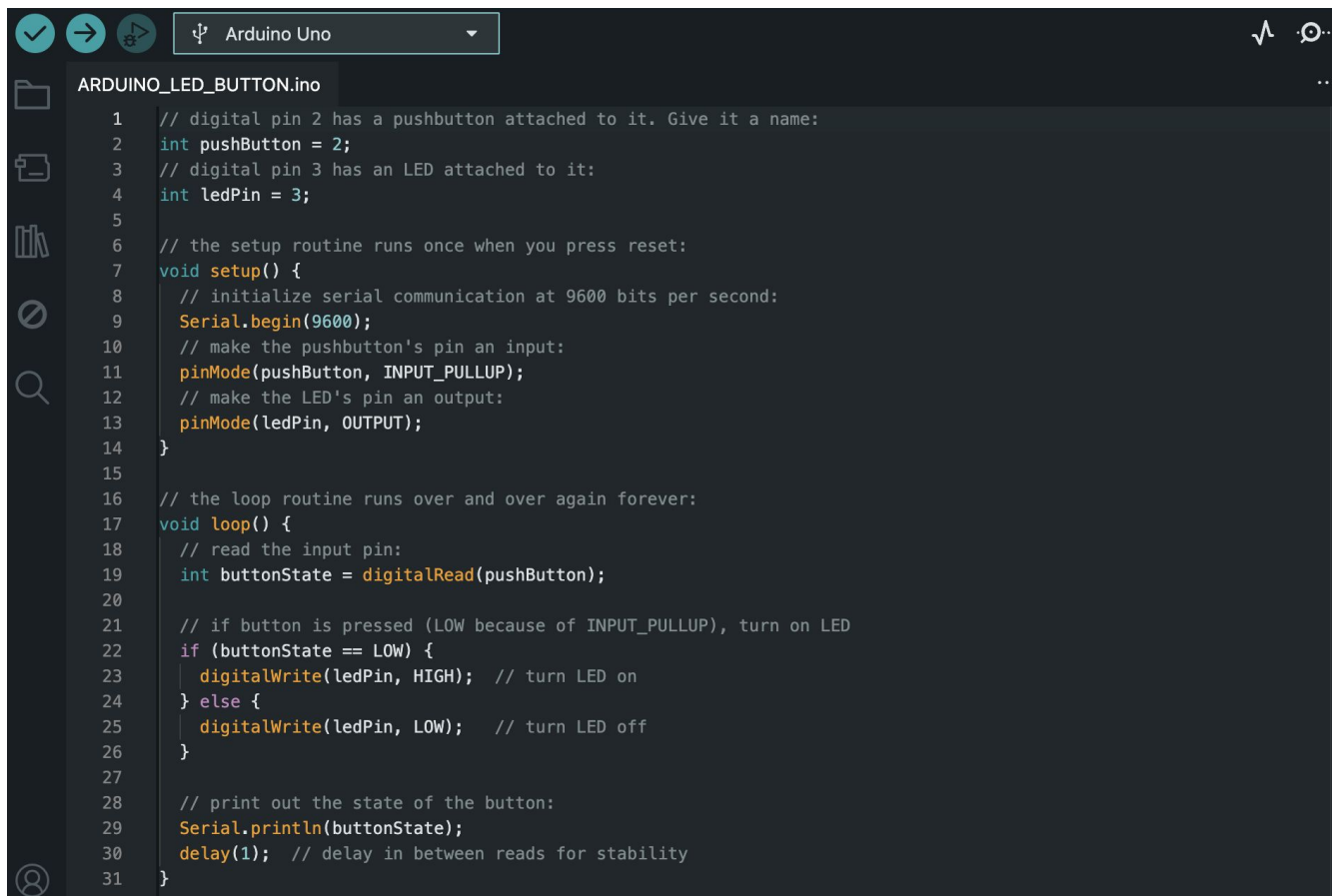
GND - Botón y LED

Pin 2 - Botón

Pin 3 - Resistencia - Led



Ejemplo. Arduino + LED + Botón



```
1 // digital pin 2 has a pushbutton attached to it. Give it a name:
2 int pushButton = 2;
3 // digital pin 3 has an LED attached to it:
4 int ledPin = 3;
5
6 // the setup routine runs once when you press reset:
7 void setup() {
8     // initialize serial communication at 9600 bits per second:
9     Serial.begin(9600);
10    // make the pushbutton's pin an input:
11    pinMode(pushButton, INPUT_PULLUP);
12    // make the LED's pin an output:
13    pinMode(ledPin, OUTPUT);
14 }
15
16 // the loop routine runs over and over again forever:
17 void loop() {
18     // read the input pin:
19     int buttonState = digitalRead(pushButton);
20
21     // if button is pressed (LOW because of INPUT_PULLUP), turn on LED
22     if (buttonState == LOW) {
23         digitalWrite(ledPin, HIGH); // turn LED on
24     } else {
25         digitalWrite(ledPin, LOW); // turn LED off
26     }
27
28     // print out the state of the button:
29     Serial.println(buttonState);
30     delay(1); // delay in between reads for stability
31 }
```


Ejemplo. Arduino + LED + Potenciómetro

Conexiones:

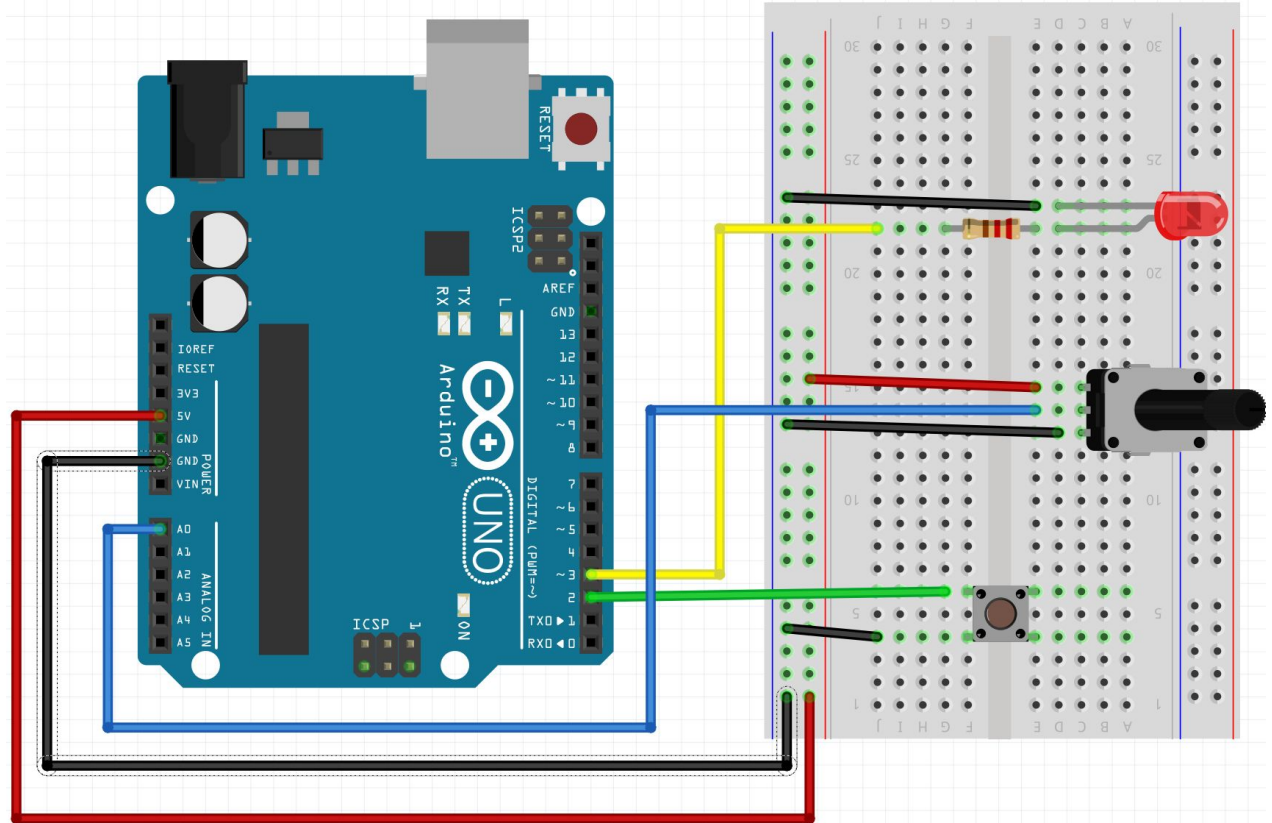
GND - Botón, LED y Pote

Pin 2 - Botón

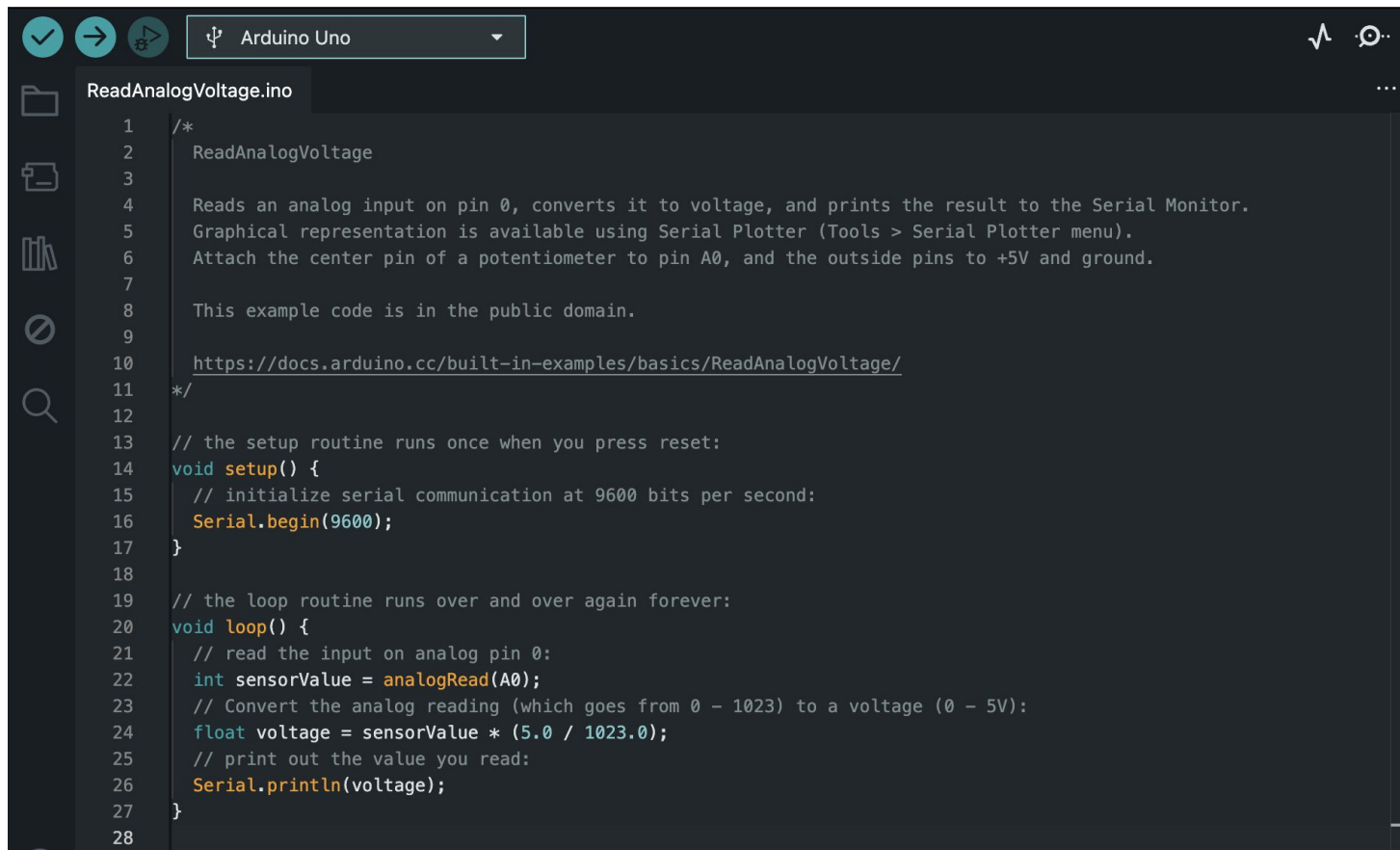
Pin 3 - Resistencia - Led

Pin A0 - Pin central Pote

+5v - Pote

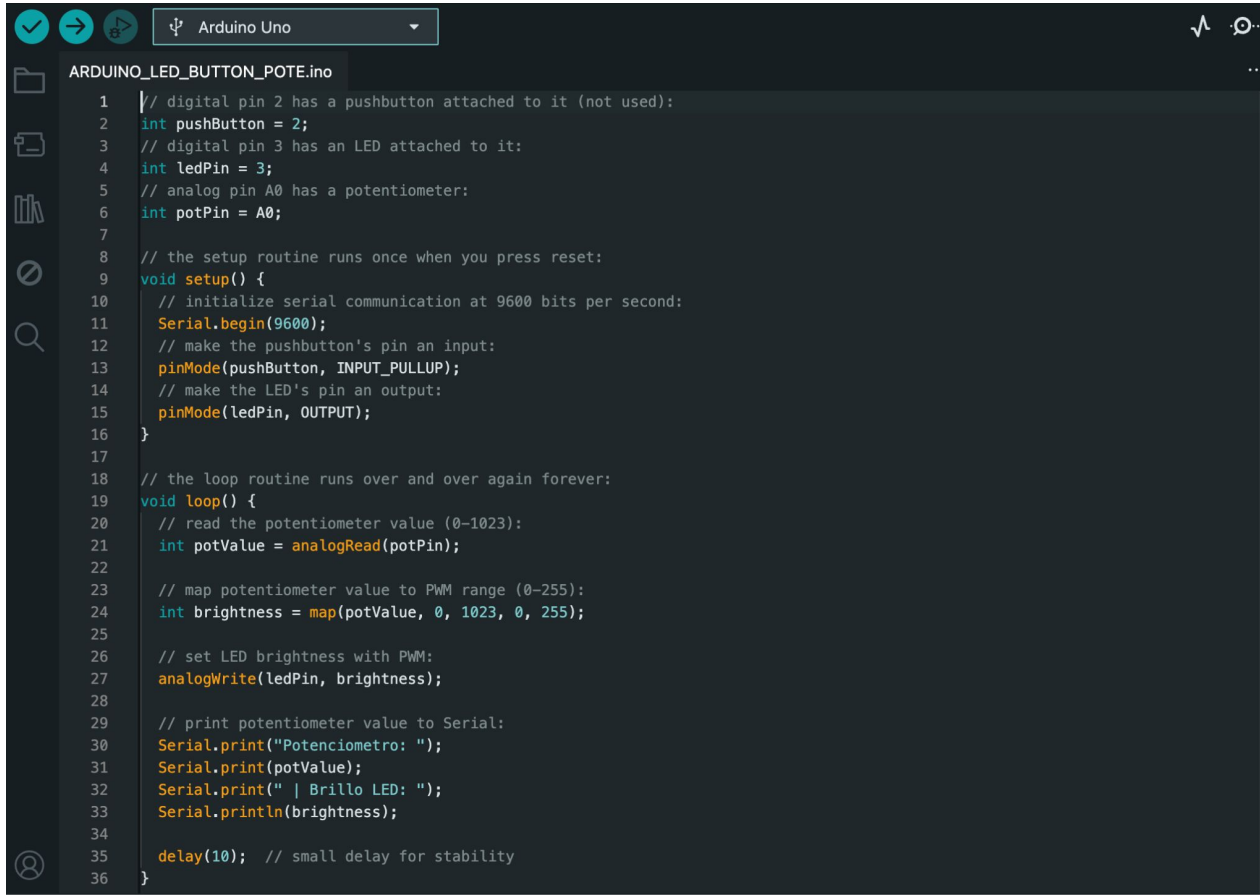


Ejemplo: Leer potenciómetro.



```
1  /*
2    ReadAnalogVoltage
3
4    Reads an analog input on pin 0, converts it to voltage, and prints the result to the Serial Monitor.
5    Graphical representation is available using Serial Plotter (Tools > Serial Plotter menu).
6    Attach the center pin of a potentiometer to pin A0, and the outside pins to +5V and ground.
7
8    This example code is in the public domain.
9
10   https://docs.arduino.cc/built-in-examples/basics/ReadAnalogVoltage/
11  */
12
13  // the setup routine runs once when you press reset:
14  void setup() {
15    // initialize serial communication at 9600 bits per second:
16    Serial.begin(9600);
17  }
18
19  // the loop routine runs over and over again forever:
20  void loop() {
21    // read the input on analog pin 0:
22    int sensorValue = analogRead(A0);
23    // Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5V):
24    float voltage = sensorValue * (5.0 / 1023.0);
25    // print out the value you read:
26    Serial.println(voltage);
27  }
28
```

Ejemplo: LED + Botón + Pote (Brillo)



```
ARDUINO_LED_BUTTON_POTE.ino
1 // digital pin 2 has a pushbutton attached to it (not used):
2 int pushButton = 2;
3 // digital pin 3 has an LED attached to it:
4 int ledPin = 3;
5 // analog pin A0 has a potentiometer:
6 int potPin = A0;
7
8 // the setup routine runs once when you press reset:
9 void setup() {
10   // initialize serial communication at 9600 bits per second:
11   Serial.begin(9600);
12   // make the pushbutton's pin an input:
13   pinMode(pushButton, INPUT_PULLUP);
14   // make the LED's pin an output:
15   pinMode(ledPin, OUTPUT);
16 }
17
18 // the loop routine runs over and over again forever:
19 void loop() {
20   // read the potentiometer value (0-1023):
21   int potValue = analogRead(potPin);
22
23   // map potentiometer value to PWM range (0-255):
24   int brightness = map(potValue, 0, 1023, 0, 255);
25
26   // set LED brightness with PWM:
27   analogWrite(ledPin, brightness);
28
29   // print potentiometer value to Serial:
30   Serial.print("Potenciometro: ");
31   Serial.print(potValue);
32   Serial.print(" | Brillo LED: ");
33   Serial.println(brightness);
34
35   delay(10); // small delay for stability
36 }
```

Ejemplo. Arduino + LED + Botón

Conexiones:

GND - Botón, LED y Pote

+5v - Pote

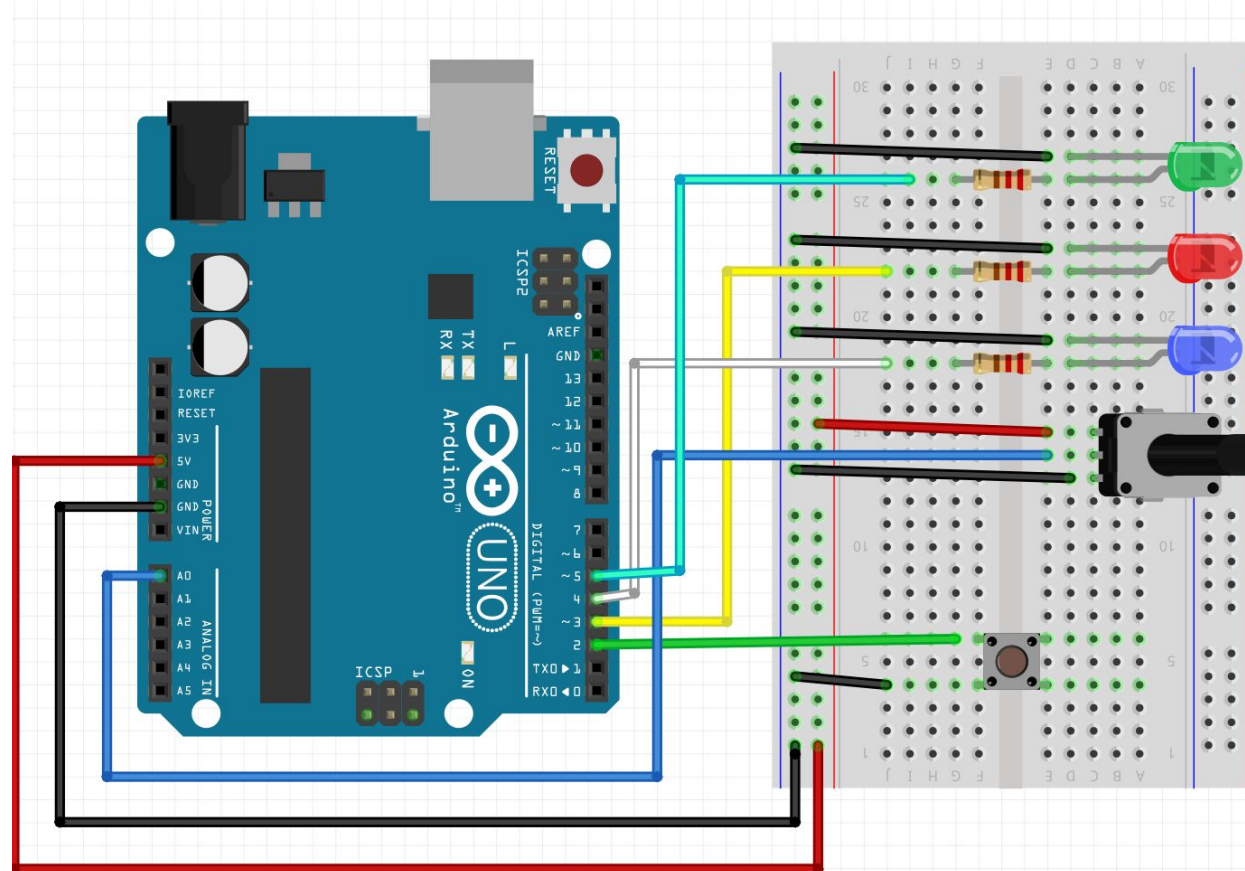
Pin 2 - Botón

Pin 3 - Resistencia - Led1

Pin 4 - Resistencia - Led2

Pin 5 - Resistencia - Led3

Pin A0 - Pin central Pote



Programación de Sensores y kit base

Librería LCD - Hola Mundo

Protocolos de comunicación: I2C

RTC en Serial y en LCD

Almacenamiento de datos en SD: SPI (ver tutorial, formato sd, etc)

Integración kit base

Lista de componentes

Uso de librerías en Arduino

¿Qué son?

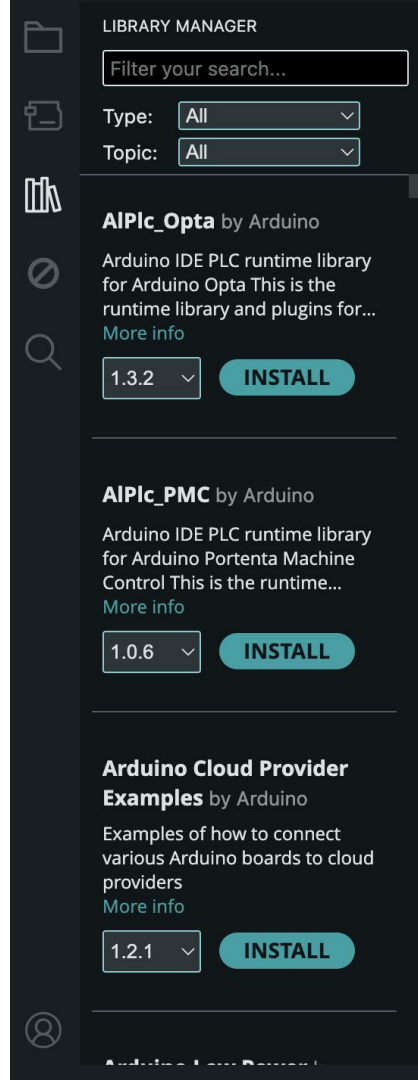
- Conjunto de funciones y clases ya programadas.
- Permiten reutilizar código sin empezar desde cero.

¿Para qué sirven?

- Controlar sensores y actuadores.
- Manejar pantallas, motores, módulos WiFi, etc.

Ventajas

- Ahorro de tiempo.
- Código más ordenado.
- Mayor fiabilidad.



Uso de las librerías en Arduino

¿Cómo se usan?

- Se las incluye en el principio del programa

```
#include <LiquidCrystal_I2C.h>
```

Internas o Externas

- **Internas:** Se encuentran dentro del Arduino IDE en la sección Library Manager o Gestor de Librerías
- **Externas:** Se descargan como archivo .zip y son agregadas desde Sketch/Include Library/ Add .zip library

Protocolos de comunicación Arduino

¿Qué es un protocolo de comunicación?

- Conjunto de reglas que permiten que dispositivos intercambien datos.

Principales protocolos de comunicación en Arduino:

- UART (Serial)
 - Pines Tx - Rx
 - ej: Comunicación con PC
- I2C
 - Pines SDA - SCL
 - Permite múltiples dispositivos. Cada uno con su ID
- SPI
 - Comunicación rápida
 - 4 líneas (MOSI, MISO, SCK, CS)
 - Ej: Memoria SD o Pantallas complejas

Kit Base: Display LCD 16x02

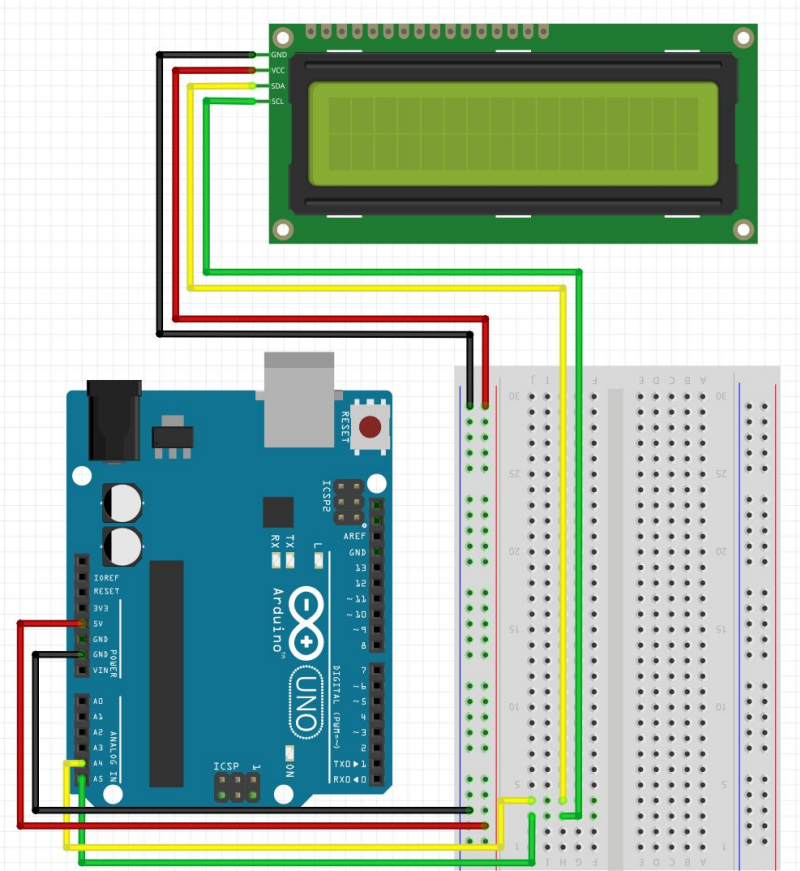
Conexiones:

GND - GND LCD

+5v - VCC LCD

Pin A4 - SDA LCD

Pin A5 - SCL LCD



Arduino Uno

HelloWorld.ino

```
1  #include <LiquidCrystal_I2C.h>
2
3  LiquidCrystal_I2C lcd(0x27,20,4); // set the LCD address to 0x27 for a 16 chars and 2 line display
4
5  void setup()
6  {
7      lcd.init();           // initialize the lcd
8      // Print a message to the LCD.
9      lcd.backlight();
10     lcd.setCursor(3,0);
11     lcd.print("Hello, world!");
12     lcd.setCursor(2,1);
13     lcd.print("Ywrobot Arduino!");
14     lcd.setCursor(0,2);
15     lcd.print("Arduino LCM IIC 2004");
16     lcd.setCursor(2,3);
17     lcd.print("Power By Ec-yuan!");
18 }
19
20
21 void loop()
22 {
23 }
24
```

Kit Base: Display LCD 16x02 + RTC

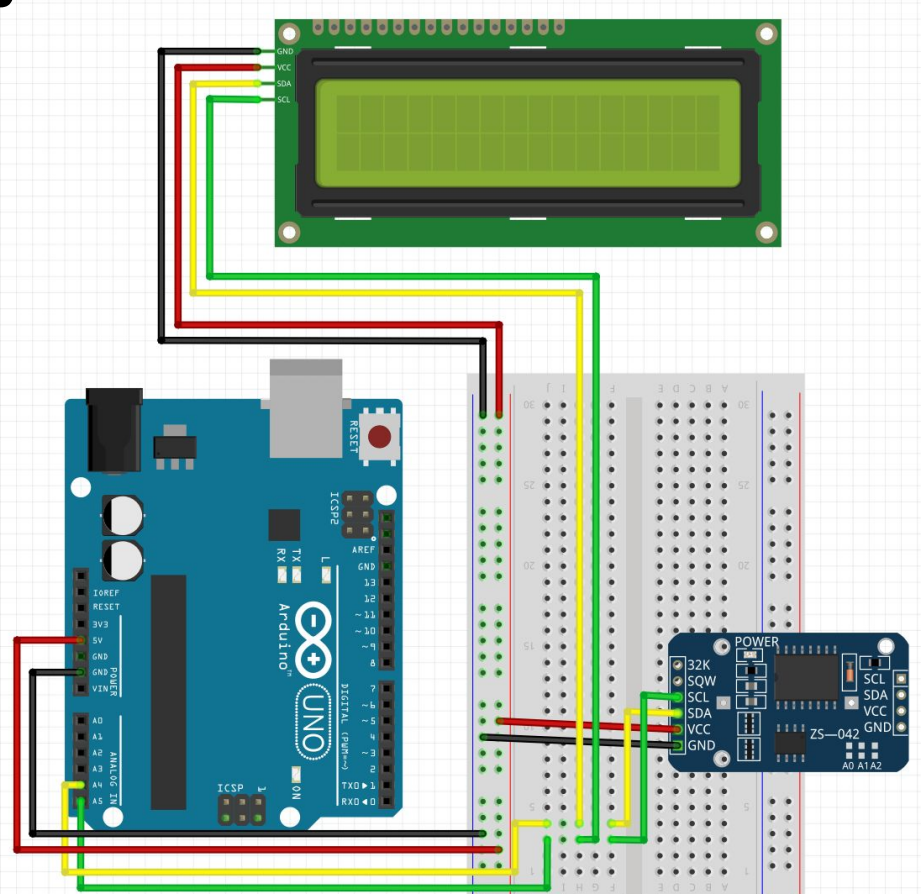
Conexiones:

GND - GND LCD y RTC

+5v - VCC LCD y RTC

Pin A4 - SDA LCD y RTC

Pin A5 - SCL LCD y RTC



Kit Base completo: Display LCD 16x02 + RTC + SD

Conexiones:

GND - GND LCD, RTC y SD

+5v - VCC LCD, RTC y SD

Pin A4 - SDA LCD, RTC y SD

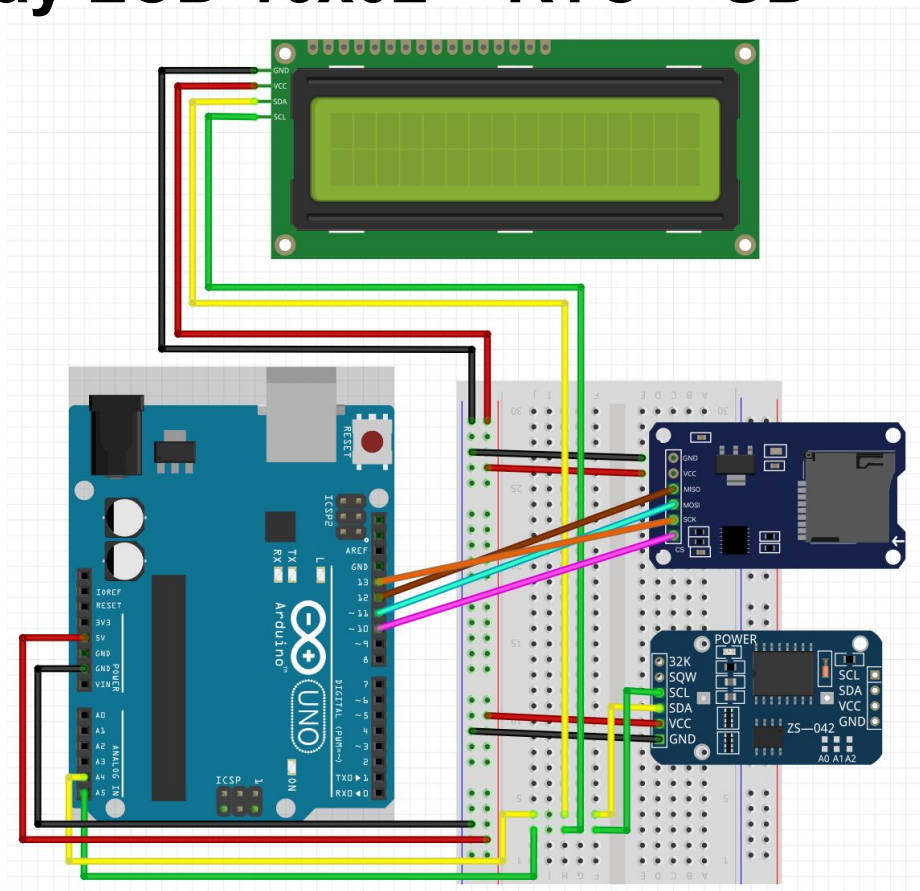
Pin A5 - SCL LCD, RTC y SD

Pin 10 - CS SD

Pin 11 - MOSI SD

Pin 12 - MISO SD

Pin 13 - SCK SD



Kit H2O completo: Base + Sensores

Conexiones:

Pin Arduino - Pin Componente

GND - GND: LCD, RTC y SD

+5v - VCC: LCD, RTC y SD

A0 - Out Turbidez

A1 - Out TDS

A2/A3 - pH / DO / EC

A4 - SDA: LCD, RTC y SD

A5 - SCL: LCD, RTC y SD

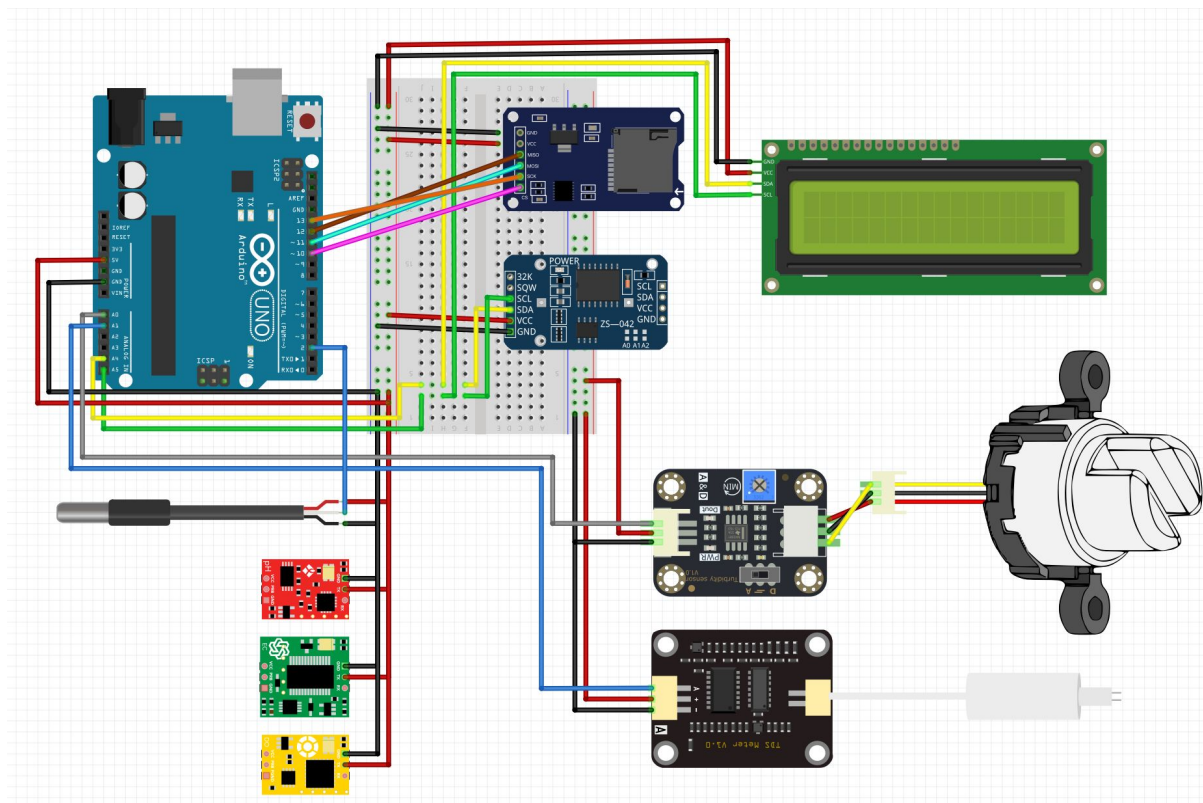
2 - Temperatura

10 - CS SD

11 - MOSI SD

12 - MISO SD

13 - SCK SD



Kit ARQ completo: Base + Sensores

Conexiones:

Pin Arduino - Pin Componente

GND - GND: LCD, RTC y SD

+5v - VCC: LCD, RTC y SD

A0 - Out MQ7

A1 - Out MQ2

A2 - Out Microfono

A4 - SDA: LCD, RTC, SD, Temp/Hum y Luz

A5 - SCL: LCD, RTC, SD, Temp/Hum y Luz

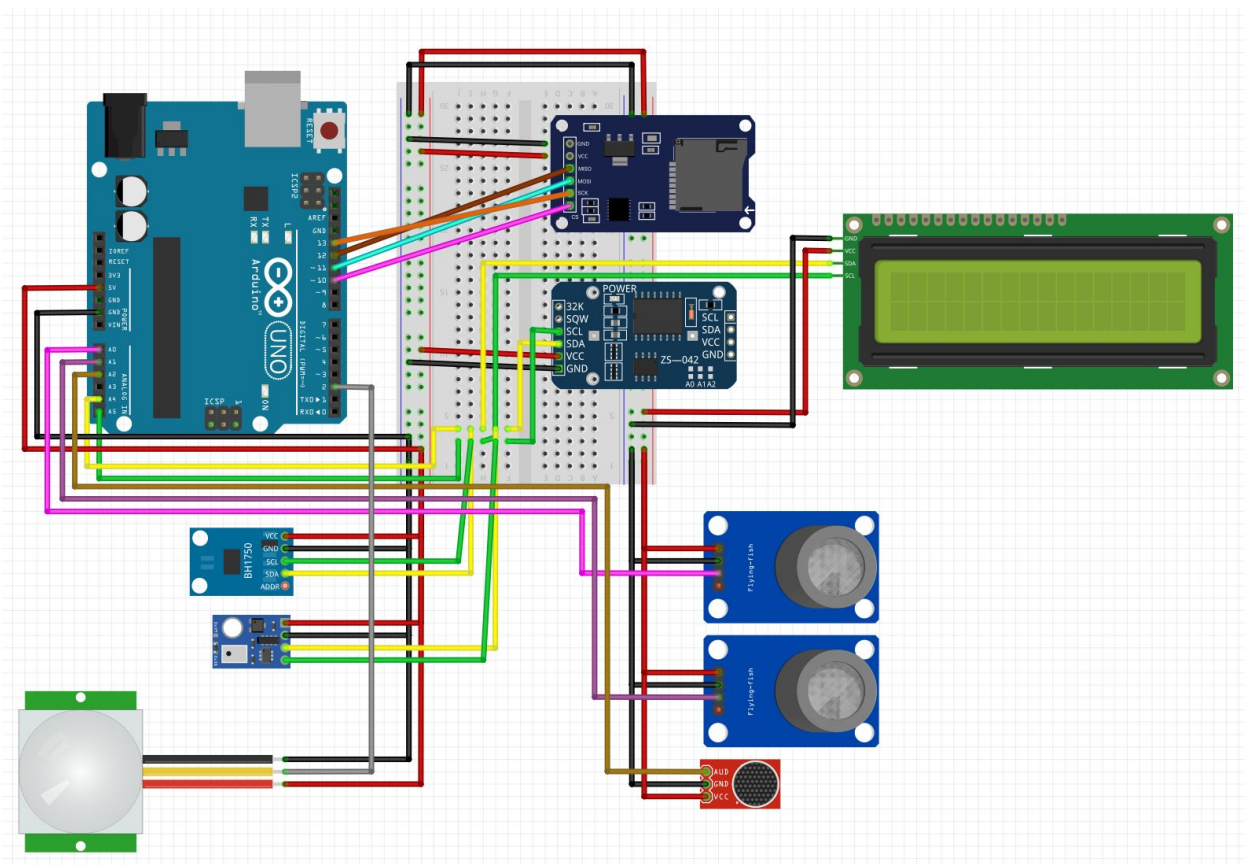
2 - Out PIR

10 - CS SD

11 - MOSI SD

12 - MISO SD

13 - SCK SD



Links

Proyectos DIY

https://www.instagram.com/p/C7zmX9cPL0g/?img_index=1

<https://www.envirodiy.org/>

<https://www.instructables.com/>

<https://randomnerdtutorials.com/>

<https://naylampmechatronics.com/content/6-tutoriales-y-proyectos-con-arduino>

Podcasts:

Inspirados x la naturaleza: <https://open.spotify.com/show/0WxxsBKP7gyRXXIWtj8fj4?si=7b55719573194263>

ECOS: <https://open.spotify.com/episode/51oE2yOXGkAtdrfCy3PyVE?si=6d9f19bef6b94225>

Proyectos Artísticos/Audiovisuales:

<https://vistprojects.com/>

<https://www.labverde.com/>

<https://www.dinalab.net/>

Proyectos afines:

<https://forum.openhardware.science/>

<https://www.instagram.com/ehys.unsam/>

+ Links

LoRa

<https://www.catsensors.com/es/lorawan/tecnologia-lora-y-lorawan>

<https://www.youtube.com/watch?v=YQ7aLHCTeeE&t=21s>

<https://www.thethingsnetwork.org/>

Recolección de datos

<https://grafana.com/>

Github:

https://github.com/nicolassagarias/PUCE_formacion



20 elementos para el **diseño** eficaz de **prompts** útiles en cualquier modelo de **IA**

1



Especifica el tono que desea, ya sea formal, relajado, educativo o persuasivo.

2



Define la estructura del resultado: ensayo, puntos, esquema, conversación.

3



Indica si deseas que adopte una perspectiva particular: experto, crítico, académico, etc.

4



Especifica el propósito del resultado: informar, persuadir, redactar.

5



Proporciona información o datos para afinar la respuesta. Por ejemplo: tendencias recientes, datos, hechos, etc.

6



Determina el rango del tema por abordar: amplio, específico, centrado en un tema...

7



Señala las palabras o frases que deben estar presentes (depende de tu objetivo y contexto)

8



Indica si hay un límite de palabras que no deban ser usadas o límite de caracteres.

9



Proporciona ejemplos o una estructura. Ejemplo: "Sigue el estilo de un informe técnico."

10



Informa sobre el público al que va dirigido. Ejemplo: estudiantes, adultos, etc.

11



Especifica en qué lenguaje deseas la respuesta si no es el lenguaje del prompt.

12



Menciona las opiniones a considerar. Por ejemplo: puntos de vista conservadores y liberales, etc.

13



Solicita los contraargumentos más comunes o desde una perspectiva específica.

14



Informa si hay términos técnicos o argot a incluir o evitar. Ejemplo: "Usa terminología médica."

15



Solicita el uso de comparaciones o ejemplos para aclarar ideas. Ejemplo: "Usa analogías con la naturaleza."

16



Solicita descripciones de gráficos o imágenes para buscarlas o crearlas.

17



Solicita frases, ideas y obras específicas (¡Debes hacer double check!)

18



Utiliza plantillas para diseñar prompts como ROCEF o PROMPTER

19



Solicita a la IA mejorar tu prompt integrando elementos específicos

20



Prueba y refina tus prompts constantemente para crear una librería personal

Prompts - IA

La nueva forma de programar



La variable como una caja

¿Qué es una Variable?

Una variable es un elemento de nuestro sketch que guarda un determinado contenido. Ese contenido (el valor de la variable) se podrá modificar en cualquier momento de la ejecución del sketch. Lo podemos imaginar como un recipiente: al recipiente le podemos dar un nombre y podemos elegir su tamaño.

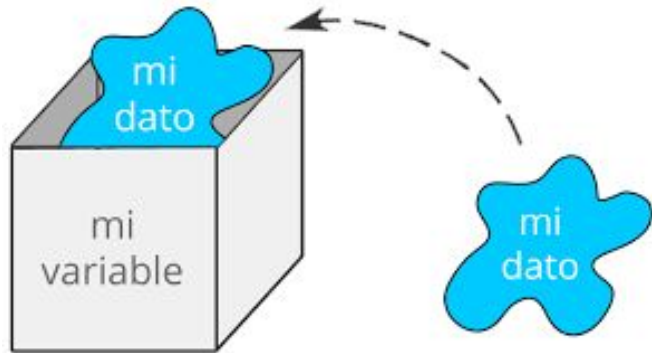
Declaración e inicialización: Declaras una vez - Usas muchas.

Ejemplo 1

```
int nombreVariable1, nombreVariable2; //Crea 2 variables de tipo int
```

```
long nombreVariable; //Crea una variable de tipo long
```

```
float nombreVariable = 2.1; //Crea una variable tipo float
```



Asignación

Para cambiar el valor de una variable, que esté o no inicializada, usaremos el operador de igualdad “=” de acuerdo al siguiente ejemplo.

Ejemplo 2

```
nombreVariable = 22; //Se asigna 22 a la variable nombreVariable
```

```
x = y + 25; //Se asigna el valor de la variable y + 25 a la variable x
```

```
valorPrevio = valorActual; //El valor de la variable valorActual es copiado a la variable  
valorPrevio
```

¿Qué es el ámbito de una variable?

El ámbito de una variable las define como globales o locales, las variables globales son las son definidas antes y fuera de las secciones de void setup(), void loop() u otra función en nuestro sketch. Y las locales son todas las que están definidas dentro de alguna función. Las variables globales reservan un espacio de memoria RAM permanentemente, y las locales lo hacen solo mientras se ejecute la función, liberando el espacio de memoria al terminarse de ejecutar la función. Por esto, las variables globales las podemos modificar desde cualquier punto del sketch, y las locales solo dentro de las funciones dentro de las cuales está declarada.

Nombre	Tamaño	Rango	Ejemplo
boolean	1 bit	1 o 0	boolean bitEncendido=0;
byte	8 bit	0 a 255	byte valor=20;
char	8 bit	-128 a 127	char código='B';
int	16 bit	-32,768 a 32,767	int val1=30000;
long	32 bit	-2,147,483,648 a 2,147,483,647	long suma=100000;
float	32 bit	-3.4028235E+38 a 3.4028235E+38	float constanteA = 5.235;
unsigned char	8 bit	0 a 255	unsigned char letra1='A';
unsigned int	16 bit	0 a 65,535	unsigned int sensor=50000;
unsigned long	32 bit	0 a 4,294,967,295	unsigned long conteo10=1;

Operadores aritméticos

=	operador de asignación
+	Operador suma
−	Operador resta
*	Operador multiplicación
/	Operador división
%	Operador módulo

Operadores compuestos

Símbolo	Ejemplo y descripción
++	x++; // incrementa x en uno y retorna el valor anterior de x
—	x− ; // decrementa x en uno y retorna el valor anterior de x
+=	x += y; // equivalente a la expresión x = x + y;
-=	x -= y; // equivalent to the expresión x = x − y;
*=	x *= y; // equivalente a la expresión x = x * y;
/=	x /= y; // equivalente a la expresión x = x / y;
%=	x %= y; // equivalente a la expresión x = x % y;